

Project Proposal:

Synthesis of Program Execution Traces for Training Machine Learning Models in Compiler-Based Tasks, in Particular Register Allocation Task

Litvinov Michael

National Research University Higher School of Economics
Saint Petersburg School of Computer Science, Physics and Technology
Saint Petersburg, Russia
malitvinov_1@edu.hse.ru

Abstract—In recent years, the integration of AI into various domains of IT has been rapidly advancing, and compilers are no exception. Large corporations are actively applying Reinforcement Learning to optimize compilers. Various frameworks, such as MLGO [1] or CompilerGym [2], have been created to integrate ML into compiler infrastructure, particularly LLVM [3].

The main bottleneck for wider application is the problem of collecting training data. This data is represented by program execution traces. Obtaining them using dynamic instrumentation takes enormous time and requires significant computational resources [4]. This paper presents the TraceSynthesizer framework, which enables trace collection, hot path analysis, statistics collection, and code embedding generation, as well as the ability to synthesize traces from source code without running actual programs. The approach is based on an LLVM plugin for control flow graph extraction, profile information, and embedding generation (MIR2VEC, IR2VEC [5]), as well as DynamoRIO [6] for collecting actual traces. The research includes the use and testing of various trace synthesis techniques, ranging from a naive probabilistic generator based solely on profile information to a complex hierarchical RL model using various code embeddings.

The successful development of this framework will overcome the bottleneck of time-consuming training trace collection. This will pave the way for broader adoption of ML in new optimizations and expand the range of problems solved using ML in compilers.

Index Terms—Compilers, Trace synthesis, LLVM, DynamoRIO, MLGO, PGO, MIR2VEC, IR2VEC, RNN, Hierarchical RL

I. INTRODUCTION

In the AI compiler environment, research focuses on optimizing two parameters: source code size and code performance. While code size optimization is relatively straightforward (e.g., measuring the size of the compiled binary or a specific section like BSS), performance optimization is a more complex endeavor. It is possible to compile a program, run it, and measure its execution time; alternatively, one can run the code, collect an execution trace, and analyze its patterns. However, for large programs, this is an extremely time-consuming and resource-intensive process, which poses a significant challenge for the entire field [3].

This issue is particularly acute in a well-known problem: register allocation for performance. To train the RL model, which acts as an advisor for GreedyRegisterAllocator in MLGO [1], the team at Google uses a complex and fragile mechanism for collecting such traces: obtaining a single trace, disabling certain passes in the LLVM pipeline, and then attempting to match one with another. This process is inherently fragile, which greatly limits the applicability of this model. If it were possible to quickly synthesize traces based on source code, profile information, embeddings, and other data, then this fragile process could be replaced with something more reliable and universal. This would open up access to a variety of transformations that modify the CFG, expand the domain, and generally have a positive impact on the industry by removing a bottleneck.

The main aim of this research is to carry out a thorough analysis and comparison of trace synthesis techniques. Besides, a prototype TraceSynthesizer infrastructure for trace generation will be developed, which is planned to be later incorporated into the MLGO ecosystem. In order to achieve the stated goal, the following aims have been laid out:

- 1) Defining what an execution trace is and deciding on which profiling tool to use.
- 2) Establishing a framework for gathering actual traces and performing simple feature extraction in code.
- 3) Initially developing a basic probabilistic trace generator based on PGO.
- 4) Adding various source code embeddings to the infrastructure.
- 5) Implementing RNNs for gaining the semantic context.
- 6) Implementing a hierarchical RL model for coding at a high-level structure.
- 7) Validation, evaluation, and comparative analysis of the results.

II. LITERATURE REVIEW

The comprehensive survey by Ashouri et al. [1] on compiler autotuning notes that "Iterative compilation is the most com-

monly used exploration technique for the compiler optimization field... However, this approach is expensive and should be combined with intelligent search algorithms, such as machine learning techniques.” The authors identify two key issues: the selection and ordering of optimizations, and they emphasize that the primary research method is iterative compilation — the process of repeatedly compiling and profiling a program. However, as noted in the review, iterative compilation is expensive and requires significant computational resources. This bottleneck has become the main driver for the application of ML.

The largest and most successful project for implementing ML at the moment is MLGO [1], a project developed by Google. It has successfully implemented an infrastructure for using ML models in an industrial compiler, LLVM [3]. The authors consider the problem of inlining for size and register allocation for performance. However, as mentioned above, the problem of collecting data for the register allocation task remains an open challenge. The authors use DynamoRIO [6] for real trace extraction, followed by a complex and brittle method of obtaining a real trace once and then overlaying it on new code at each training step to speed up the process.

Recent work has explored bypassing physical code execution. For instance, the CompilerDream architecture [7] simulates intermediate representation changes and rewards, avoiding actual compilation. However, in this paper, the authors attempt to simulate higher-level phenomena – the results of IR changes. This approach confirms the relevance of replacing physical data collection with synthetic data, and the reward smoothing methods used there are applicable to solving the problem of sparse feedback in RL.

In the context of data synthesis, the ShIOEnv framework [8], designed for synthesizing Bash commands and simulating their behavior in an isolated environment, is of particular interest. The authors formulate the synthesis process as an RL problem, where an agent sequentially selects command arguments. They use a grammatical constraint on the action space, and the command grammar acts as a “supervisor” that permits only valid token sequences. Similar to this framework, in our problem, the role of such a grammar is played by a control flow graph, which defines all valid sequences of basic blocks.

One of the prerequisites for trace synthesis is the development of code embeddings such as IR2Vec [5] and ProGraML [9]. They are already integrated into LLVM and are actively used for model training, creating the technological foundation for our approach.

Another prerequisite is the development of hierarchical machine learning models [10]. The FeUdal Networks framework [11] introduces a hierarchy where a “Manager” operates at a low temporal resolution and a “Worker” operates at a high resolution (every tick). This architecture naturally maps to working with Call Context (high-level) and instructions within a block (low-level). This method has already been used in academic papers related to ML in compilers [12].

This literature review demonstrates that, despite significant

advances in the application of ML to code optimization, the problem of data collection remains unresolved. Existing approaches to data synthesis (CompilerDream [7], ShIOEnv [8]) either address related problems or do not account for the specifics of execution traces. Advances in code representation methods (embeddings) and hierarchical RL create the prerequisites for developing an effective method for synthesizing traces. This study aims to fill this gap by proposing an approach that integrates grammatical constraints - CFG, modern code embeddings, and hierarchical RL to generate realistic execution traces necessary for training compiler models.

III. RESEARCH DESIGN AND METHODS

Before proceeding, it is necessary to formally define the concept of an execution trace within the context of this study. This is a far from trivial task. One can select the sequence of all instructions executed by the processor, filter instructions by module (for example, all instructions from a specific binary), or construct a CFG graph (at the final stage of the binary) and use only the basic block sequence as a trace. Since the first two options generate data of colossal size, the BB sequence was chosen as the target representation. The tool for obtaining the CFG was a custom-written LLVM pass, which allows for obtaining all necessary statistics (including profiling information) on the code and its CFG at the final stage of the pipeline.

Next, a profiling tool was selected. DynamoRIO [6] was chosen to maintain compatibility with MLGO [1]. DynamoRIO enables the collection of real program execution traces, which are then aligned with the statically obtained CFG. These traces serve as the ground truth for the trajectory synthesis task.

A. Baseline Probabilistic Generator

Our infrastructure implements a naive generator based solely on static information as a baseline. Algorithm: The generator performs a random walk (from the entry point) through the control flow graph. At each step, while in the basic block BB_i , the next edge is chosen randomly, but with a probability proportional to the weights from the profile information. The process continues until a terminal block is reached or until a specified trace length limit is exceeded. Limitations: This approach does not take into account context-dependent relationships between branches (e.g., the correlation between the decision at the beginning of the function and the decision at its end), resulting in the generation of many “noisy” paths that are not encountered in real runs.

B. Code Representation and Sequence Modeling with RNNs

Before formulating the problem in terms of reinforcement learning, the problem of input data encoding must be addressed. The program state at the time of trace generation cannot be reduced solely to the identifier of the current basic block – the agent must understand the semantics of the code within the block and take into account the history of the path already traversed.

For each basic block, a set of features is extracted from the final CFG:

- **Static features:** a histogram of instruction types (similar to Autophase in CompilerDream [7]).
- **Semantic embeddings:** using pretrained models such as MIR2Vec, IR2Vec [5] or ProGraML [9] to obtain dense vector representations encoding the semantics of the instructions within a block and its structural role in the graph.

Since an execution trace is a sequence of basic blocks, it is natural to use an RNN for its modeling. In this paper, we consider LSTM as the underlying architecture for processing sequences. LSTM is able to retain information about decisions made early in the trace, which is critical for modeling context-sensitive branching (for example, when the choice in a function’s epilogue depends on the branch taken in the prologue).

In the supervised pre-training phase, the LSTM is trained to predict the next basic block BB_{t+1} based on the sequence of previous blocks $[BB_0, \dots, BB_t]$ and their embeddings. This allows the model to capture the underlying transition patterns present in the ground-truth traces. Lighter-weight GRUs or, given sufficient data, Transformer decoders can be used as an alternative to LSTM. However, in this prototype, LSTM was chosen as the most balanced solution.

C. Reinforcement Learning Formulation

An RNN model pretrained on real traces is capable of generating plausible continuations, but it optimizes cross-entropy (the accuracy of predicting the next token) rather than global properties of the trace (e.g., the frequency of hitting “cold” regions or conformity to the PGO distribution). To solve this problem, a transition to a reinforcement learning formulation is made.

1) *Formalization as an MDP:* The trace synthesis process is considered as a Markov Decision Process, defined by a tuple $\langle S, A, P, R, \gamma \rangle$: **State (S):** The state space includes:

- The current basic block BB_t , represented by its embedding.
- The hidden state of the LSTM h_t , which encodes the path history and semantic context.
- Call context encoded as a separate vector.
- A counter of current block visits for loop resolution.

Action (A): The action space represents the choice of one of the admissible edges of the CFG leading out of the current block BB_t . The agent cannot choose an edge that is not present in the graph. Thus, the CFG acts as a “grammar” guiding the synthesis.

Transition function (P): Deterministic and defined by the CFG structure. If the agent chooses action a_t , leading to block BB_{t+1} , the next state is BB_{t+1} .

Reward function (R): The reward can be calculated as a measure of the similarity of the generated trace to the real one. In the simplest case, this can be the negative log-loss of the next block prediction (which brings RL closer to supervised learning). In a more complex setting, the reward can take into

account global statistics (e.g., the KL divergence of the block visit frequency distribution with PGO).

2) *Training Algorithm:* Proximal Policy Optimization is proposed as the base RL algorithm. PPO has proven itself in compiler-based problems [1], [2] and is robust to hyperparameters. The policy $\pi_\theta(a_t|s_t)$ and value function $V_\phi(s_t)$ will be implemented based on the LSTM encoder pretrained in the previous stage. This allows the agent to start not from scratch, but with a pre-trained understanding of trace structure.

D. Evaluation Metrics

The following metrics are used to compare the quality of the generated traces:

- **Distribution Comparison:** KL divergence between the distribution of paths (or block visit frequencies) generated by the model and the same distribution obtained from real traces (DynamoRIO).
- **Hot Path Accuracy:** The degree of overlap between the most frequent block sequences in generated and real traces.
- **Generation Speed:** Comparison of the time required to generate 1,000 traces using the synthetic method versus the time to collect 1,000 traces using DynamoRIO.

IV. ANTICIPATED RESULTS

This work is expected to yield the following scientific and practical results:

TraceSynthesizer infrastructure prototype: A functional software system integrated with the LLVM ecosystem, including:

- 1) A module for accurately extracting CFGs and statistics from code exactly before the AsmPrinter stage in LLVM.
- 2) An interface for collecting and processing real traces from DynamoRIO.
- 3) A utility for visualizing CFGs, highlighting hot paths, and overlaying ground-truth traces onto the graph.
- 4) Implementation of a probabilistic (PGO-based) trace generator (baseline).
- 5) Implementation of an RL agent (at least a flat PPO) for trace synthesis.

Proof of Concept for HRL: The hierarchical model (FuN/h-DQN) is expected to demonstrate statistically significant superiority over the probabilistic baseline and flat RL in terms of KL divergence and hot path accuracy. Such a result would confirm that accounting for hierarchical code structure (call context, loops) is critical for realistic trace synthesis.

Training Dataset: A collection of synthetic traces generated for a suite of benchmarks (e.g., SPEC or cBench), which can be used to train downstream models such as register allocation policies.

Quantitative Speedup: Trace synthesis is expected to be orders of magnitude faster than direct trace collection (minutes versus hours/days), thereby eliminating a key bottleneck in the ML compiler pipeline.

Word Count: 2175

REFERENCES

- [1] M. Trofin, Y. Qian, E. Brevdo, Z. Lin, K. Choromanski, and D. Li, "MLGO: a Machine Learning Guided Compiler Optimizations Framework," *arXiv preprint arXiv:2101.04808*, 2021.
- [2] C. Cummins et al., "CompilerGym: robust, performant compiler optimization environments for AI research," in *2022 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2022, pp. 92-105.
- [3] C. Lattner and V. Adve, "LLVM: A compilation framework for lifelong program analysis & transformation," in *International Symposium on Code Generation and Optimization (CGO)*, 2004, pp. 75-86.
- [4] A. H. Ashouri, W. Killian, J. Cavazos, G. Palermo, and C. Silvano, "A survey on compiler autotuning using machine learning," *ACM Computing Surveys (CSUR)*, vol. 51, no. 5, pp. 1-42, 2018.
- [5] S. VenkataKeerthy, R. Aggarwal, S. Jain, M. S. Desarkar, R. Upadrasta, and Y. N. Srikanth, "IR2Vec: A Framework for LLVM IR based Scalable Program Embeddings," *ACM Transactions on Architecture and Code Optimization (TACO)*, vol. 17, no. 4, pp. 1-27, 2020.
- [6] D. Bruening, T. Garnett, and S. Amarasinghe, "An infrastructure for adaptive dynamic optimization," in *International Symposium on Code Generation and Optimization (CGO)*, 2003, pp. 265-275.
- [7] C. Deng, J. Wu, N. Feng, J. Wang, and M. Long, "CompilerDream: Learning a Compiler World Model for General Code Optimization," in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '25)*, 2025.
- [8] J. Ragsdale and R. Boppana, "ShIOEnv: A Command Evaluation Environment for Grammar-Constrained Synthesis and Execution Behavior Modeling," *arXiv preprint arXiv:2505.18374*, 2025.
- [9] C. Cummins, Z. V. Fisches, T. Ben-Nun, T. Hoefler, M. F. O'Boyle, and H. Leather, "ProGraML: A Graph-based Program Representation for Data Flow Analysis and Compiler Optimizations," in *International Conference on Machine Learning (ICML)*, 2021, pp. 2244-2253.
- [10] T. D. Kulkarni, K. Narasimhan, A. Saeedi, and J. Tenenbaum, "Hierarchical Deep Reinforcement Learning: Integrating Temporal Abstraction and Intrinsic Motivation," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 29, 2016.
- [11] A. S. Vezhnevets et al., "FeUdal Networks for Hierarchical Reinforcement Learning," in *International Conference on Machine Learning (ICML)*, 2017, pp. 3540-3549.
- [12] VenkataKeerthy, S. and Jain, Siddharth and Kundu, Anilava and Aggarwal, Rohit and Cohen, Albert and Upadrasta, Ramakrishna, "RL4ReAl: Reinforcement Learning for Register Allocation," *arXiv preprint arXiv:2204.02013*, 2023.